

Additional Pretraining and Embedding Results Supporting the iMLM+ITC Selection

Instruction-Reconstruction Evaluation

To probe what each model learned beyond token-prediction accuracy, entire PTX instructions are masked and each model must recover all constituent tokens from surrounding instruction context alone which is a strictly harder test than standard masked-token recovery, designed to expose whether the model captures instruction structure or merely token statistics.

Table 1: Top-1/Top-5 accuracy by token frequency bucket, instruction-reconstruction evaluation.

Frequency bucket	MLM		iMLM		iMLM+ITC	
	Top-1	Top-5	Top-1	Top-5	Top-1	Top-5
High	72.6	88.1	97.7	99.7	97.1	99.5
Medium	50.4	64.2	85.3	94.3	74.9	88.6
Low	47.4	58.4	80.5	91.5	66.1	82.5
Overall	67.2	82.0	94.4	98.2	91.1	96.4

MLM degrades sharply as token frequency drops (72.6%→47.4% Top-1), approaching random guessing on rare tokens, since it can exploit local token co-occurrence rather than instruction structure. iMLM and iMLM+ITC remain robust across all frequency regimes (80%+ Top-1 even on low-frequency tokens), confirming they internalize PTX instruction syntax/semantics rather than surface statistics.

Embedding-Space Separation

We probe the frozen encoders’ representations along two axes: (1) **address-space separation**, whether `global.*` vs. `shared.*` memory operations occupy distinct embedding regions (Addr. Inter = cosine similarity between class centroids, lower is better; Sep. gap = intra-class similarity minus $2\times$ Addr. Inter, higher is better); and (2) **memory-vs-compute separation** (Mem/Comp = cosine similarity between memory-op and arithmetic-op centroids, lower is better).

Table 2: Embedding-space separation metrics for the three pretrained encoders.

Model	Addr. Inter↓	Intra _{global}	Intra _{shared}	Sep. gap↑	Mem/Comp↓
MLM	0.788	0.728	0.711	−0.137	0.740
iMLM	0.788	0.720	0.713	−0.143	0.749
iMLM+ITC	0.334	0.673	0.755	+0.760	0.455

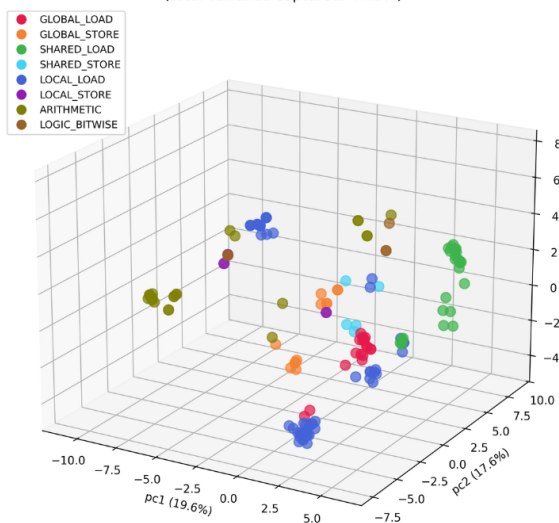
MLM and iMLM both yield *negative* separation gaps: cross-class embeddings are closer than same-class ones, i.e. neither objective organizes the space by address space or by memory-vs-compute role which is expected, since neither rewards instruction-type discrimination. iMLM+ITC changes this qualitatively: Addr. Inter drops from 0.788 to 0.334, Sep. gap becomes strongly positive (+0.760), and Mem/Comp drops from ≈ 0.74 to 0.455. Figure 1 visualizes this directly: PCA of instruction embeddings colored by functional class shows tightly separated, well-formed clusters (e.g. global loads, shared loads, arithmetic) for iMLM+ITC, versus overlapping, poorly separated clusters for iMLM alone.

Objective Selection

MLM achieves the highest raw token-accuracy but fails under instruction-level reconstruction and yields no meaningful class separation. iMLM substantially improves structural understanding. iMLM+ITC adds strong embedding-space separation by instruction type on top of iMLM’s structural understanding, at a modest cost in token-prediction accuracy, since the ITC gradient reallocates encoder capacity toward instruction-type discrimination which is the property the downstream fine-tuning stage exploits to relate code structure to contention-induced slowdown. We therefore select iMLM+ITC as the pretrained backbone for fine-tuning.

IMLM

PCA: PTX instruction embeddings by ITC class
(total variance captured: 47.3%)



IMLM+ITC

PCA: PTX instruction embeddings by ITC class
(total variance captured: 59.8%)

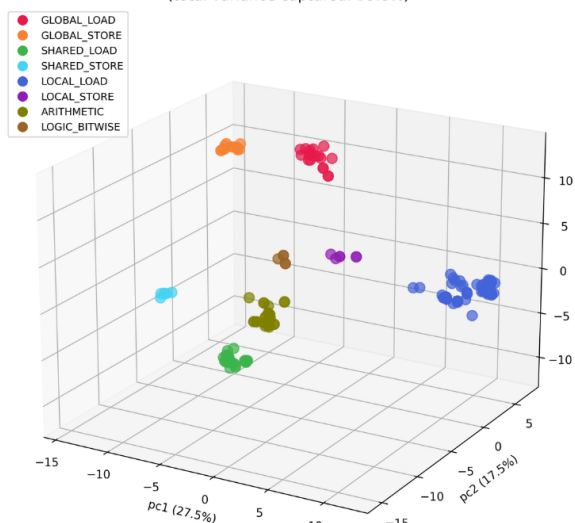


Figure 1: PCA of PTX instruction embeddings by functional class: iMLM (left) vs. iMLM+ITC (right). iMLM+ITC captures more variance (59.8% vs. 47.3%) and forms visibly tighter, better-separated clusters per instruction class.